

Name: Smruti Ranjan
Registration No : 2025CA096
Subject: Data Structure(Lab)

Assignment (Linked) List Theory

1. What is Linked lists? What type of memory allocation is referred for Linked lists? what is traversal in linked lists?

Linked List

A linked list is a **linear data structure** where elements (nodes) are stored in **non-contiguous memory locations**. Each node contains:

- **Data**
- **Pointer** (address of the next node)

Memory Allocation

- ❖ **Dynamic memory allocation** (using malloc / new).

Traversal

Traversal is the process of **visiting each node exactly once** starting from the head node till NULL.

2. Describe what is Node in link list? And name the types of Linked Lists and Explain each?

Node

A node is the basic unit of a linked list.

```
struct Node {  
    int data;  
    struct Node* next;  
};
```

Types of Linked Lists

1. **Singly Linked List**
Each node points to the next node.
2. **Doubly Linked List**
Each node has two pointers: previous and next.
3. **Circular Linked List**
Last node points back to the first node.
4. **Circular Doubly Linked List**
Combination of circular + doubly.

3. What will you prefer to use a singly or a doubly linked lists for traversing through a list of elements?

I will preferred **Doubly Linked List**.

Reason: Traversal is possible in **both directions** (forward & backward).

4. Explain the application of linked list with example in detail.

Applications

- Dynamic memory management
- Implementation of stacks & queues
- Polynomial manipulation
- Undo/Redo operations
- Music playlist

Example:

Music playlist where songs can be added or removed dynamically.

5. Mention the steps to insert data at the starting of a singly linked list? How to reverse a singly linked list?

Insert at Beginning

Steps

1. Create a new node
2. New node → next = head
3. Head = new node

```
void insertAtStart(struct Node** head, int x) {
    struct Node* newNode = malloc(sizeof(struct Node));
    newNode->data = x;
    newNode->next = *head;
    *head = newNode;
}
```

Reverse a Singly Linked List

```
struct Node* reverse(struct Node* head) {
    struct Node *prev = NULL, *curr = head, *next;
    while(curr) {
        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```

6. Mention what is the difference between Linear Array and Linked List?

Basis of Comparison	Linear Array	Linked List
Definition	Collection of elements stored in contiguous memory locations	Collection of nodes stored in non-contiguous memory locations
Memory Allocation	Static or dynamic , but size must be known in advance	Dynamic memory allocation at runtime
Size	Fixed size once declared	Dynamic size , can grow or shrink
Memory Utilization	May cause wastage if size is overestimated	Efficient use of memory (no wastage of unused space)
Accessing Elements	Direct / Random access using index ($O(1)$)	Sequential access only ($O(n)$)
Insertion	Costly, requires shifting elements ($O(n)$)	Easy, only pointer changes ($O(1)$ if position known)
Deletion	Costly, requires shifting elements ($O(n)$)	Easy, just adjust pointers ($O(1)$ if position known)
Memory Overhead	No extra memory required	Extra memory needed for pointer(s)
Cache Friendliness	Cache friendly due to contiguous memory	Not cache friendly
Implementation	Simple and easy to implement	Comparatively complex
Searching	Binary search possible (if sorted)	Binary search not possible
Traversal Time	Faster traversal	Slower traversal
Flexibility	Less flexible	More flexible
Examples	<code>int arr[10];</code>	Nodes connected using pointers
Applications	Static data, lookup tables	Dynamic data, stacks, queues, playlists

7. How to remove loops in a linked list (or) what are fast and slow pointers used for?

Fast & Slow pointers (Floyd's Cycle Detection)

- Slow moves 1 step
 - Fast moves 2 steps
- If they meet → **loop exists**

Used to:

- Detect cycle
- Find start of loop
- Remove loop

8. Compare Linked lists and Dynamic Arrays.

Basis	Linked List	Dynamic Array (e.g., vector, ArrayList)
Memory Allocation	Non-contiguous memory allocation	Contiguous memory allocation
Size	Fully dynamic (grows/shrinks node by node)	Dynamic but resizes in blocks
Access Method	Sequential access only	Random access using index
Insertion (Middle)	Efficient – $O(1)$ if position known	Costly – elements must be shifted
Deletion (Middle)	Efficient – $O(1)$ if position known	Costly – shifting required
Insertion (End)	$O(1)$ (if tail pointer exists)	Amortized $O(1)$, worst $O(n)$ during resize
Deletion (End)	$O(n)$ (singly list)	$O(1)$
Memory Overhead	Extra memory for pointer(s)	No extra pointer per element
Cache Performance	Poor (nodes scattered in memory)	Good (contiguous memory)
Search Operation	$O(n)$	$O(n)$, binary search possible if sorted
Resizing Cost	No resizing overhead	Expensive when resizing occurs
Implementation	More complex	Easier to implement
Use Cases	Frequent insertions/deletions	Frequent access, indexing

9. How many pointers are required to implement a simple Linked list? Justify the answer.

One pointer per node is required to implement a simple Linked list

Justification: Only the **address of next node** is required.

10. How to Delete Alternate Nodes Of A Linked List?

```
void deleteAlternate(struct Node* head) {  
    struct Node* curr = head;  
    while(curr && curr->next) {  
        struct Node* temp = curr->next;  
        curr->next = temp->next;  
        free(temp);  
        curr = curr->next;  
    }  
}
```

11. Given a single linked list L1 which is sorted in ascending order , and another single linked list L2 which is not sorted. Write a function to print the elements of second list according to the first list . For Example if the first list is 1->2->5->7->8->, then the function should print the 1st ,2nd ,5th ,7th ,8th element of the second list.

```
#include <stdio.h>  
#include <stdlib.h>  
  
/* Structure of a node */  
struct Node {  
    int data;  
    struct Node* next;
```

```

};

/* Function to create a new node */
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

/* Function to append node at end */
void append(struct Node** head, int data) {
    struct Node* newNode = createNode(data);

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    struct Node* temp = *head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
}

/* Function to display linked list */
void display(struct Node* head) {
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

/* Function to print elements of L2 according to L1 */
void printAccordingToFirst(struct Node* L1, struct Node* L2) {
    struct Node* p1 = L1;

    printf("Required elements from second list:\n");

    while (p1 != NULL) {
        int k = p1->data;
        struct Node* p2 = L2;
        int count = 1;

```

```

        while (p2 != NULL && count < k) {
            p2 = p2->next;
            count++;
        }

        if (p2 != NULL)
            printf("%d ", p2->data);

        p1 = p1->next;
    }
    printf("\n");
}

/* Main function */
int main() {
    printf("\n");
    struct Node* L1 = NULL;
    struct Node* L2 = NULL;

    /* Creating first Linked List (sorted positions) */
    append(&L1, 1);
    append(&L1, 2);
    append(&L1, 5);
    append(&L1, 7);
    append(&L1, 8);

    /* Creating second Linked List (unsorted data) */
    append(&L2, 10);
    append(&L2, 20);
    append(&L2, 30);
    append(&L2, 40);
    append(&L2, 50);
    append(&L2, 60);
    append(&L2, 70);
    append(&L2, 80);

    printf("First Linked List (Positions):\n");
    display(L1);

    printf("\nSecond Linked List (Data):\n");
    display(L2);

    printf("\n");
    printAccordingToFirst(L1, L2);
}

```

```

printf("\n");
return 0;
}

```

Output:

```

Windows PowerShell
PS C:\Users\piyus\Desktop\DataStructure\lab4\Assignment 4> gcc Q11.c
PS C:\Users\piyus\Desktop\DataStructure\lab4\Assignment 4> ./a.exe

First Linked List (Positions):
1 -> 2 -> 5 -> 7 -> 8 -> NULL

Second Linked List (Data):
10 -> 20 -> 30 -> 40 -> 50 -> 60 -> 70 -> 80 -> NULL

Required elements from second list:
10 20 50 70 80

PS C:\Users\piyus\Desktop\DataStructure\lab4\Assignment 4> |

```

- Write a function to insert a node just before and just after a node pointed to by a pointer p, without using the pointer start.

```

#include <stdio.h>
#include <stdlib.h>

/* Node structure */
struct Node {
    int data;
    struct Node* next;
};

/* Create a new node */
struct Node* createNode(int x) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = x;
    newNode->next = NULL;
    return newNode;
}

/* Append node at end (used only to create List) */
void append(struct Node** head, int x) {
    struct Node* newNode = createNode(x);

    if (*head == NULL) {

```

```

        *head = newNode;
        return;
    }

    struct Node* temp = *head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
}

/* Display Linked List */
void display(struct Node* head) {
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

/* Insert a node just AFTER pointer p */
void insertAfter(struct Node* p, int x) {
    if (p == NULL) return;

    struct Node* newNode = createNode(x);
    newNode->next = p->next;
    p->next = newNode;
}

/* Insert a node just BEFORE pointer p (without using start) */
void insertBefore(struct Node* p, int x) {
    if (p == NULL) return;

    struct Node* newNode = createNode(p->data);
    newNode->next = p->next;

    p->data = x;
    p->next = newNode;
}

/* Main function */
int main() {
    printf("\n");
    struct Node* head = NULL;

```

```

/* Creating initial Linked List */
append(&head, 10);
append(&head, 20);
append(&head, 30);
append(&head, 40);

printf("Original Linked List:\n");
display(head);

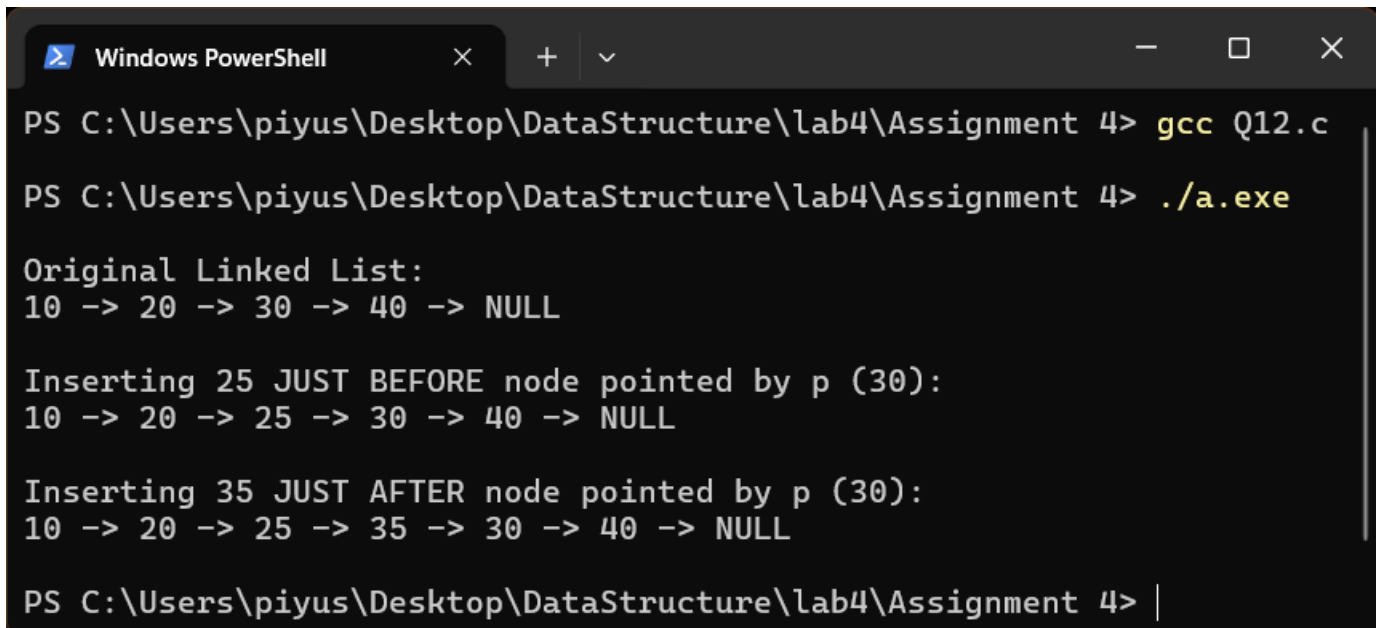
/* Pointer p pointing to node with data 30 */
struct Node* p = head->next->next;

printf("\nInserting 25 JUST BEFORE node pointed by p (30):\n");
insertBefore(p, 25);
display(head);

printf("\nInserting 35 JUST AFTER node pointed by p (30):\n");
insertAfter(p, 35);
display(head);
printf("\n");
return 0;
}

```

Output:



```

Windows PowerShell
PS C:\Users\piyus\Desktop\DataStructure\lab4\Assignment 4> gcc Q12.c
PS C:\Users\piyus\Desktop\DataStructure\lab4\Assignment 4> ./a.exe

Original Linked List:
10 -> 20 -> 30 -> 40 -> NULL

Inserting 25 JUST BEFORE node pointed by p (30):
10 -> 20 -> 25 -> 30 -> 40 -> NULL

Inserting 35 JUST AFTER node pointed by p (30):
10 -> 20 -> 25 -> 35 -> 30 -> 40 -> NULL

PS C:\Users\piyus\Desktop\DataStructure\lab4\Assignment 4> |

```

13. write a function to remove duplicates from a sorted linked list.

```
#include <stdio.h>
#include <stdlib.h>

/* Node structure */
struct Node {
    int data;
    struct Node* next;
};

/* Create new node */
struct Node* createNode(int x) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = x;
    newNode->next = NULL;
    return newNode;
}

/* Append node at end */
void append(struct Node** head, int x) {
    struct Node* newNode = createNode(x);

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    struct Node* temp = *head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
}

/* Display Linked List */
void display(struct Node* head) {
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

/* Remove duplicates from sorted Linked List */
```

```

void removeDuplicates(struct Node* head) {
    struct Node* curr = head;

    while (curr != NULL && curr->next != NULL) {
        if (curr->data == curr->next->data) {
            struct Node* temp = curr->next;
            curr->next = temp->next;
            free(temp);
        } else {
            curr = curr->next;
        }
    }
}

/* Main function */
int main() {
    printf("\n");
    struct Node* head = NULL;

    /* Creating a sorted Linked List with duplicates */
    append(&head, 10);
    append(&head, 10);
    append(&head, 20);
    append(&head, 20);
    append(&head, 20);
    append(&head, 30);
    append(&head, 40);
    append(&head, 40);

    printf("Original Sorted Linked List:\n");
    display(head);

    removeDuplicates(head);

    printf("\nLinked List After Removing Duplicates:\n");
    display(head);
    printf("\n");

    return 0;
}

```

Output:

```
Windows PowerShell
PS C:\Users\piyus\Desktop\DataStructure\lab4\Assignment 4> gcc Q13.c
PS C:\Users\piyus\Desktop\DataStructure\lab4\Assignment 4> ./a.exe

Original Sorted Linked List:
10 -> 10 -> 20 -> 20 -> 20 -> 30 -> 40 -> 40 -> NULL

Linked List After Removing Duplicates:
10 -> 20 -> 30 -> 40 -> NULL

PS C:\Users\piyus\Desktop\DataStructure\lab4\Assignment 4> |
```

14. write a function to delete the alternate node (even numbered node) from a single linked list. For Example if the list is 1->2->3->4->5->6->7 then the resulting list should be 1->3->5->7.

```
#include <stdio.h>
#include <stdlib.h>

/* Node structure */
struct Node {
    int data;
    struct Node* next;
};

/* Create new node */
struct Node* createNode(int x) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = x;
    newNode->next = NULL;
    return newNode;
}

/* Append node at end */
void append(struct Node** head, int x) {
    struct Node* newNode = createNode(x);

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    struct Node* temp = *head;
    while (temp->next != NULL)
```

```

        temp = temp->next;

    temp->next = newNode;
}

/* Display Linked List */
void display(struct Node* head) {
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

/* Delete alternate (even numbered) nodes */
void deleteAlternate(struct Node* head) {
    struct Node* curr = head;

    while (curr != NULL && curr->next != NULL) {
        struct Node* temp = curr->next;
        curr->next = temp->next;
        free(temp);
        curr = curr->next;
    }
}

/* Main function */
int main() {
    printf("\n");
    struct Node* head = NULL;

    /* Creating Linked List */
    append(&head, 1);
    append(&head, 2);
    append(&head, 3);
    append(&head, 4);
    append(&head, 5);
    append(&head, 6);
    append(&head, 7);

    printf("Original Linked List:\n");
    display(head);

    deleteAlternate(head);
}

```

```

printf("\nLinked List After Deleting Alternate Nodes:\n");
display(head);
printf("\n");

return 0;
}

```

Output:

```

Windows PowerShell
PS C:\Users\piyus\Desktop\DataStructure\lab4\Assignment 4> gcc Q14.c
PS C:\Users\piyus\Desktop\DataStructure\lab4\Assignment 4> ./a.exe

Original Linked List:
1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> NULL

Linked List After Deleting Alternate Nodes:
1 -> 3 -> 5 -> 7 -> NULL

PS C:\Users\piyus\Desktop\DataStructure\lab4\Assignment 4> |

```

15. Write a function to find out whether a single linked list is NULL terminated or contains a cycle /loop. If the list contain a cycle find the length of the cycle and length of the whole list. Find the node that causes the cycle i.e. the node at which the cycle starts. The node is pointed by two different nodes of list . Remove the cycle from the list and make it NULL terminated.

```

#include <stdio.h>
#include <stdlib.h>

/* Node structure */
struct Node {
    int data;
    struct Node* next;
};

/* Create a new node */
struct Node* createNode(int x) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = x;
    newNode->next = NULL;
    return newNode;
}

/* Append node at end */
void append(struct Node** head, int x) {

```

```

    struct Node* newNode = createNode(x);

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    struct Node* temp = *head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
}

/* Detect cycle using Floyd's algorithm */
struct Node* detectCycle(struct Node* head) {
    struct Node *slow = head, *fast = head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;

        if (slow == fast)
            return slow;    // Cycle detected
    }
    return NULL;           // No cycle
}

/* Find length of cycle */
int cycleLength(struct Node* meet) {
    int count = 1;
    struct Node* temp = meet->next;

    while (temp != meet) {
        count++;
        temp = temp->next;
    }
    return count;
}

/* Find starting node of cycle */
struct Node* findCycleStart(struct Node* head, struct Node* meet) {
    struct Node* p1 = head;
    struct Node* p2 = meet;

```

```

while (p1 != p2) {
    p1 = p1->next;
    p2 = p2->next;
}
return p1;
}

/* Remove cycle */
void removeCycle(struct Node* head, struct Node* start) {
    struct Node* temp = start;

    while (temp->next != start)
        temp = temp->next;

    temp->next = NULL;
}

/* Find length of list */
int listLength(struct Node* head) {
    int count = 0;
    while (head != NULL) {
        count++;
        head = head->next;
    }
    return count;
}

/* Display Linked List */
void display(struct Node* head) {
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

/* Main function */
int main() {
    printf("\n");
    struct Node* head = NULL;

    /* Creating Linked List */
    append(&head, 10);
    append(&head, 20);
    append(&head, 30);

```

```

append(&head, 40);
append(&head, 50);

/* Creating a cycle: Last node points to node with data 30 */
head->next->next->next->next->next = head->next->next;

struct Node* meet = detectCycle(head);

if (meet == NULL) {
    printf("The linked list is NULL terminated (No cycle).\n");
} else {
    printf("The linked list contains a CYCLE.\n");

    int clen = cycleLength(meet);
    printf("Length of cycle: %d\n", clen);

    struct Node* start = findCycleStart(head, meet);
    printf("Cycle starts at node with data: %d\n", start->data);

    removeCycle(head, start);
    printf("Cycle removed successfully.\n");

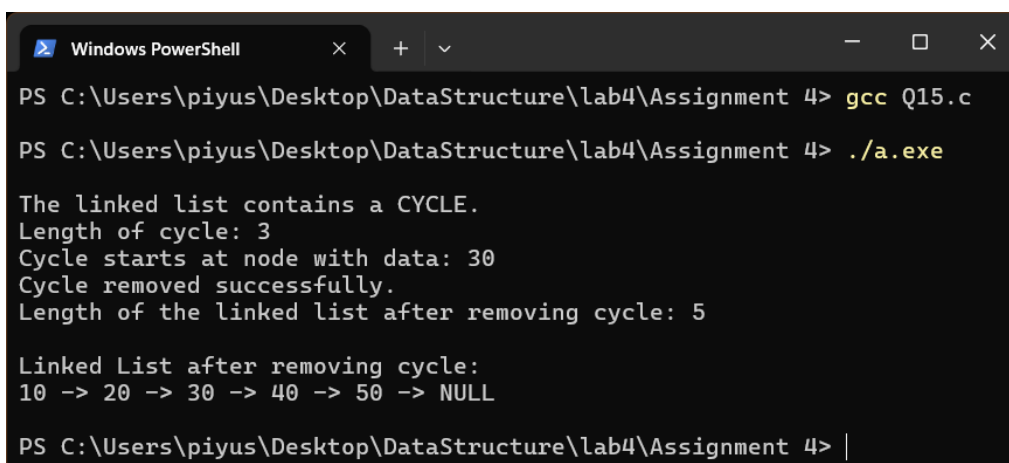
    int totalLen = listLength(head);
    printf("Length of the linked list after removing cycle: %d\n", totalLen);

    printf("\nLinked List after removing cycle:\n");
    display(head);
}
printf("\n");

return 0;
}

```

Output:



```

Windows PowerShell
PS C:\Users\piyus\Desktop\DataStructure\lab4\Assignment 4> gcc Q15.c
PS C:\Users\piyus\Desktop\DataStructure\lab4\Assignment 4> ./a.exe

The linked list contains a CYCLE.
Length of cycle: 3
Cycle starts at node with data: 30
Cycle removed successfully.
Length of the linked list after removing cycle: 5

Linked List after removing cycle:
10 -> 20 -> 30 -> 40 -> 50 -> NULL

PS C:\Users\piyus\Desktop\DataStructure\lab4\Assignment 4> |

```

16. Create a double linked list in which info part of each node contains a digit of a given number, the digits should be stored in reverse order i.e. the least significant digit should be stored in first node and most significant digit in last node. If the number is 5468132 then the linked list should be 2->3->1->8->6->4->5->. Write a function to add two numbers represented by linked list.

```
#include <stdio.h>
#include <stdlib.h>

/* Doubly Linked List Node */
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

/* Create a new node */
struct Node* createNode(int x) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = x;
    newNode->prev = NULL;
    newNode->next = NULL;
    return newNode;
}

/* Insert node at end */
void insertEnd(struct Node** head, int x) {
    struct Node* newNode = createNode(x);

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    struct Node* temp = *head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;
}

/* Create DLL from number (digits stored in reverse order) */
struct Node* createListFromNumber(long num) {
    struct Node* head = NULL;
```

```

while (num > 0) {
    int digit = num % 10;
    insertEnd(&head, digit);
    num = num / 10;
}
return head;
}

/* Display doubly linked list */
void display(struct Node* head) {
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

/* Add two numbers represented by doubly linked lists */
struct Node* addNumbers(struct Node* l1, struct Node* l2) {
    struct Node* result = NULL;
    int carry = 0;

    while (l1 != NULL || l2 != NULL || carry != 0) {
        int sum = carry;

        if (l1 != NULL) {
            sum += l1->data;
            l1 = l1->next;
        }

        if (l2 != NULL) {
            sum += l2->data;
            l2 = l2->next;
        }

        carry = sum / 10;
        insertEnd(&result, sum % 10);
    }
    return result;
}

/* Main function */
int main() {
    printf("\n");
    long num1 = 5468132;

```

```

long num2 = 789;

struct Node* L1 = createListFromNumber(num1);
struct Node* L2 = createListFromNumber(num2);

printf("First Number as Doubly Linked List:\n");
display(L1);

printf("\nSecond Number as Doubly Linked List:\n");
display(L2);

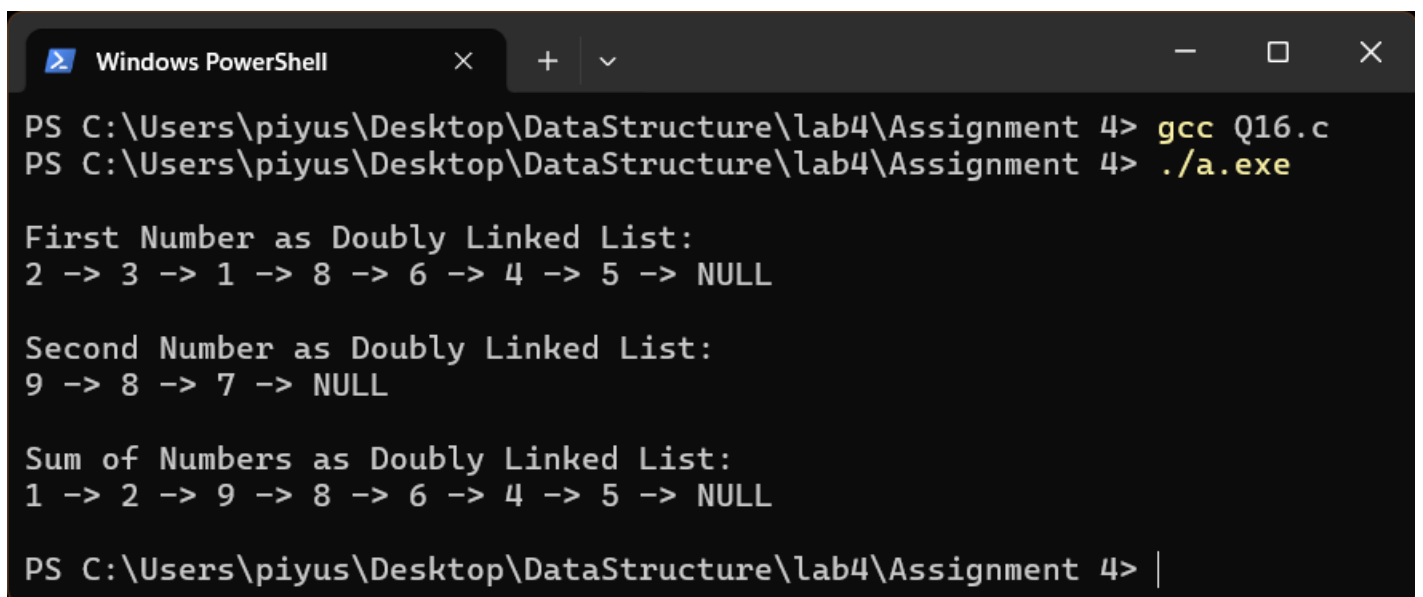
struct Node* sumList = addNumbers(L1, L2);

printf("\nSum of Numbers as Doubly Linked List:\n");
display(sumList);
printf("\n");

return 0;
}

```

Output:



```

Windows PowerShell
PS C:\Users\piyus\Desktop\DataStructure\lab4\Assignment 4> gcc Q16.c
PS C:\Users\piyus\Desktop\DataStructure\lab4\Assignment 4> ./a.exe

First Number as Doubly Linked List:
2 -> 3 -> 1 -> 8 -> 6 -> 4 -> 5 -> NULL

Second Number as Doubly Linked List:
9 -> 8 -> 7 -> NULL

Sum of Numbers as Doubly Linked List:
1 -> 2 -> 9 -> 8 -> 6 -> 4 -> 5 -> NULL

PS C:\Users\piyus\Desktop\DataStructure\lab4\Assignment 4> |

```